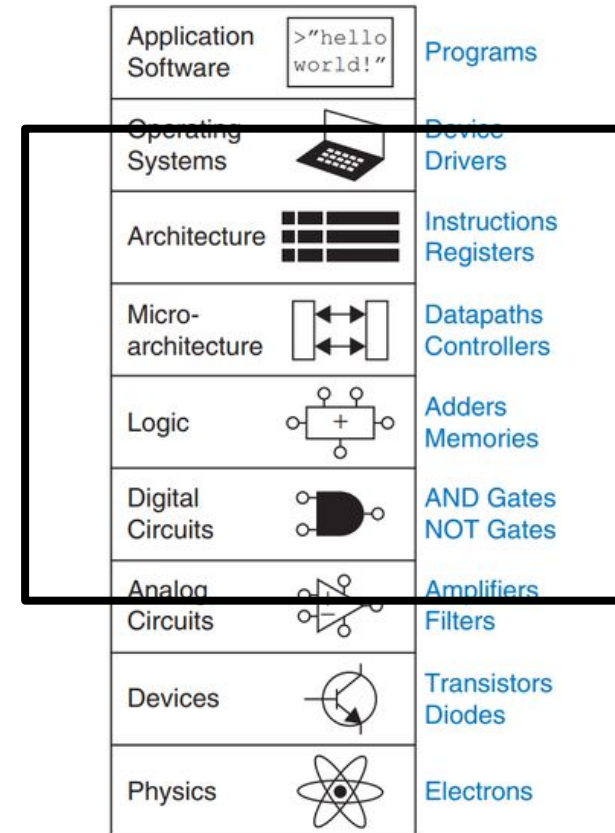


Aula 03: Fundamentos VLSI + VDMS



Revisão

- **Aula Anterior (bottom-up)**
 - Modelos / MOSFET
 - Portas Lógicas / CMOS
 - Circuitos Combinacionais
 - Circuitos Sequenciais
 - Standard Cells / PDK
 - *PPA Budgets* (LIMITE)
 - **"PPA Budgets"**



VLSI



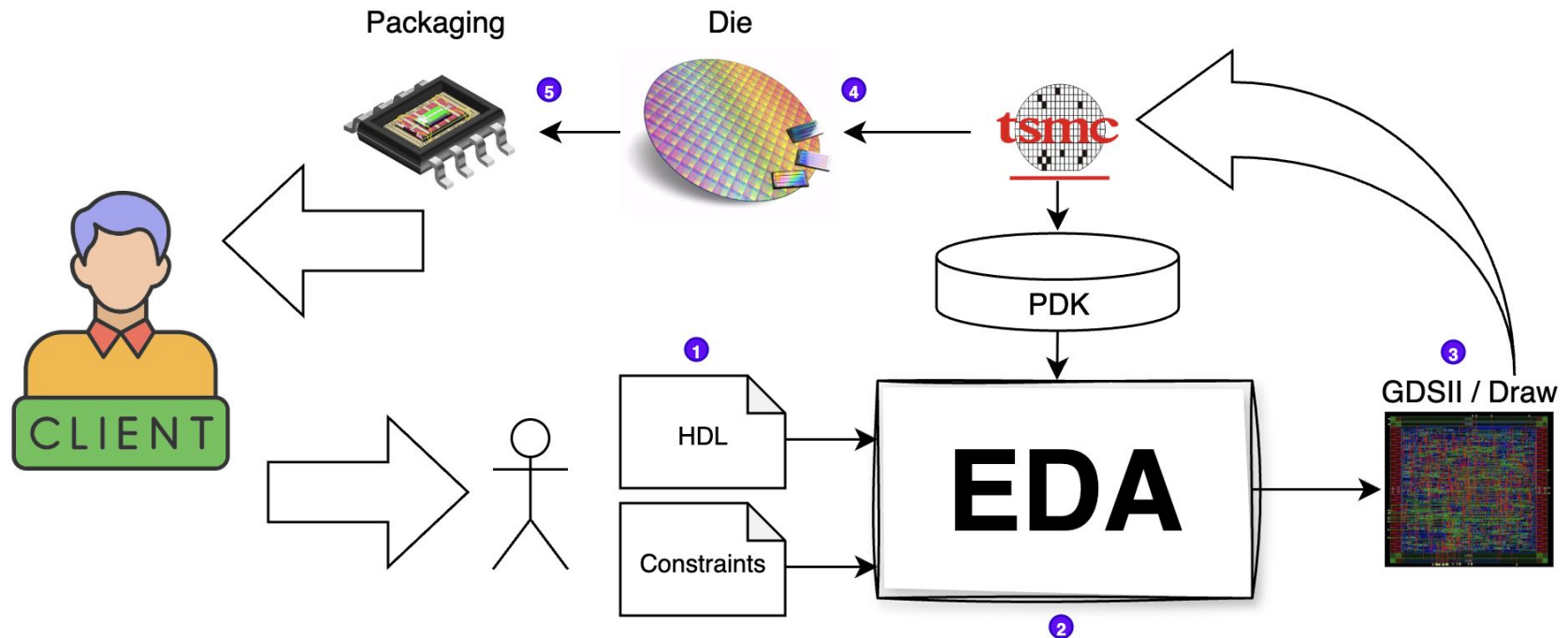
Agenda

- **VLSI Flow**

Etapas do Fluxo e Ferramentas Envolvidas na Prática

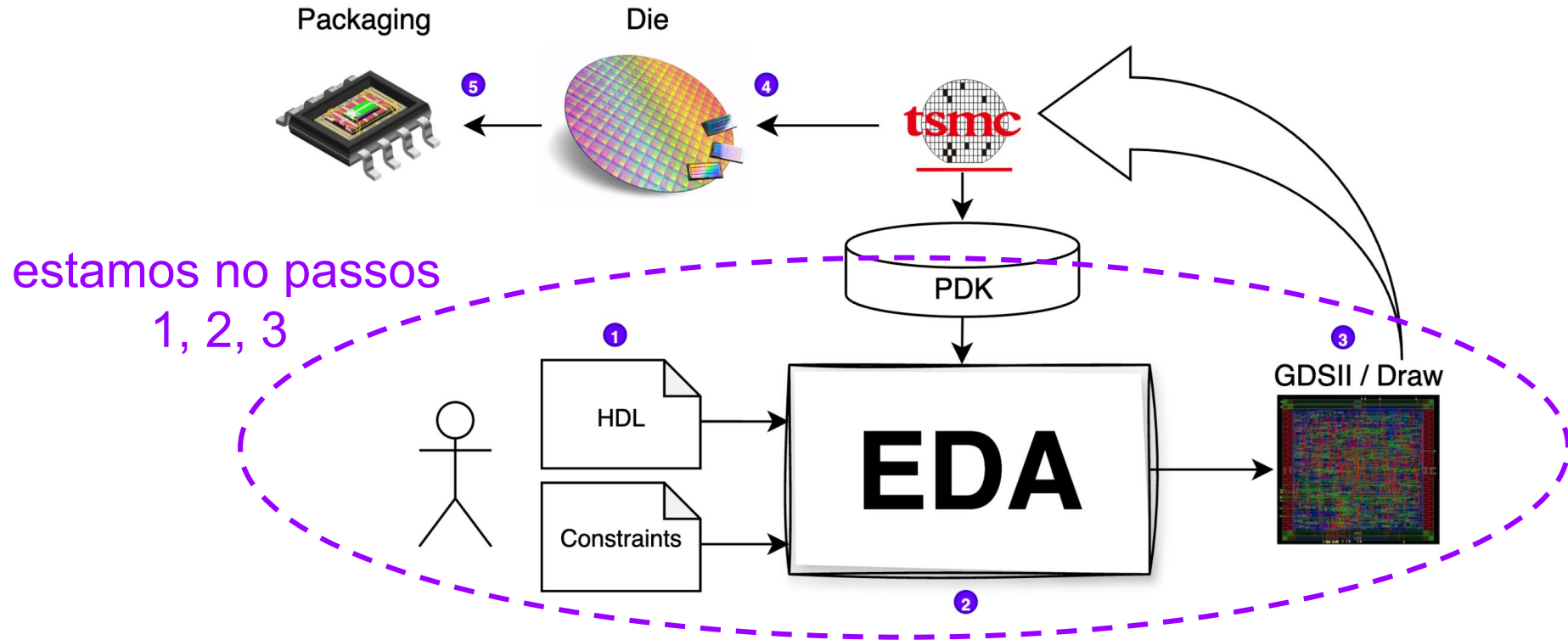
VLSI Flow

- Como tudo isso aparece no fluxo VLSI?



VLSI Flow

- Como tudo isso aparece no fluxo VLSI?

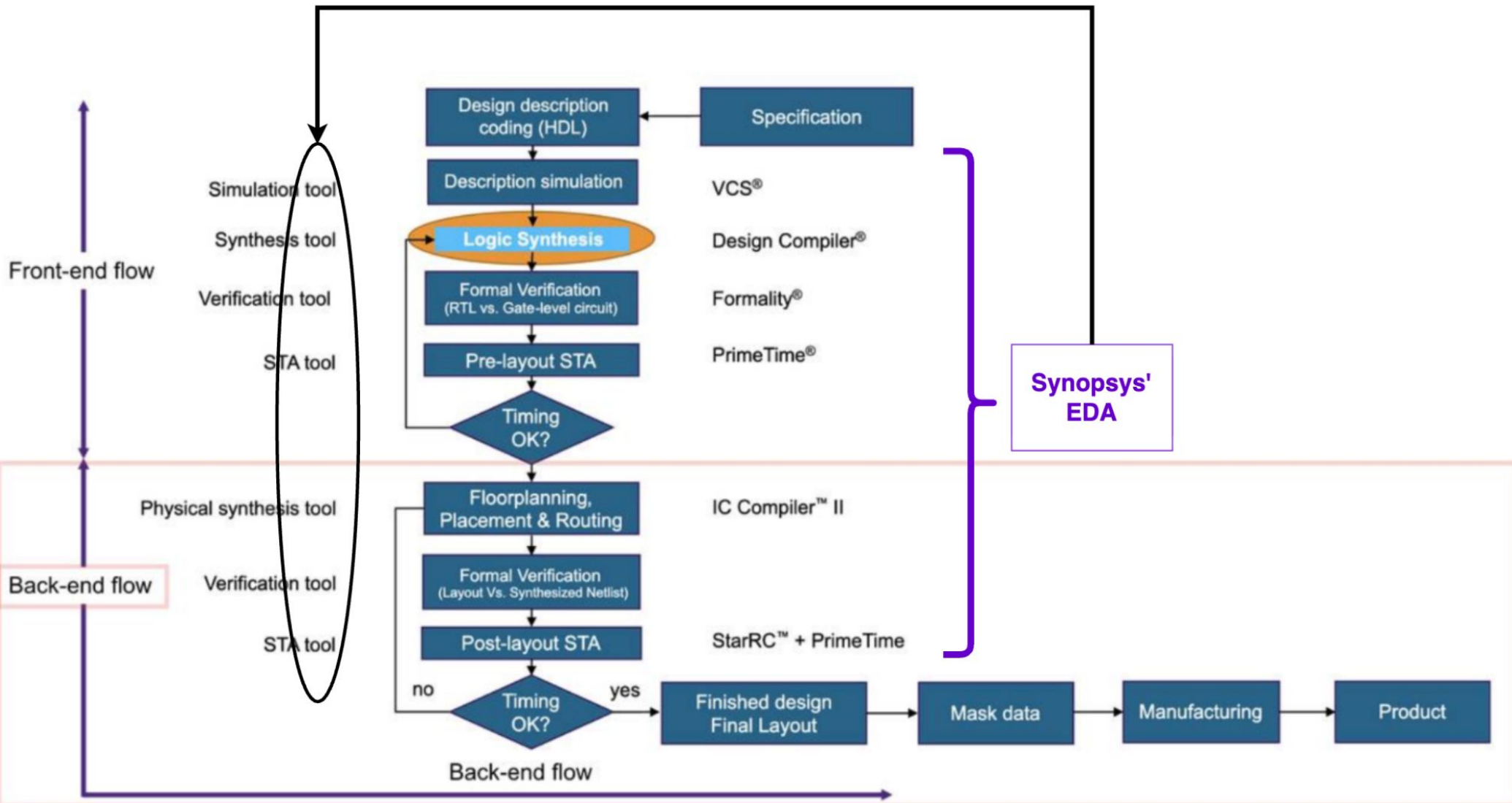




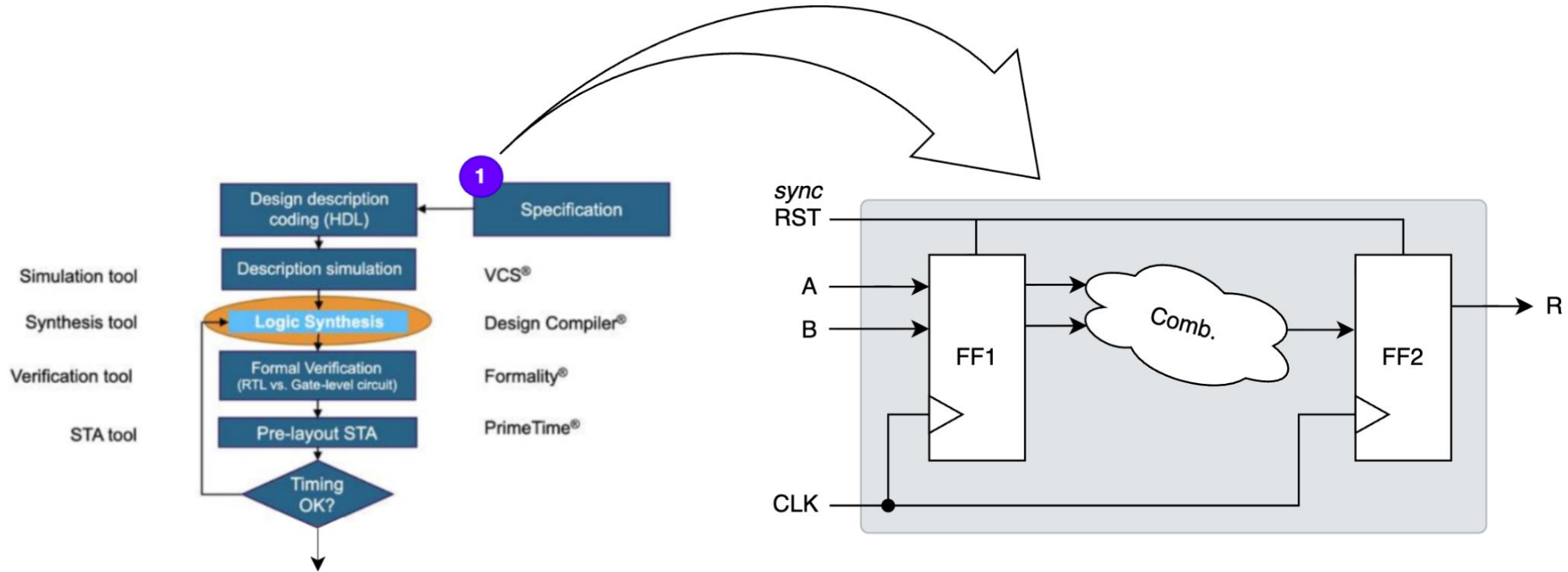
VLSI Flow

- Vamos entender o **fluxo front-end** na prática...

EDAs are tools to run the VLSI flow

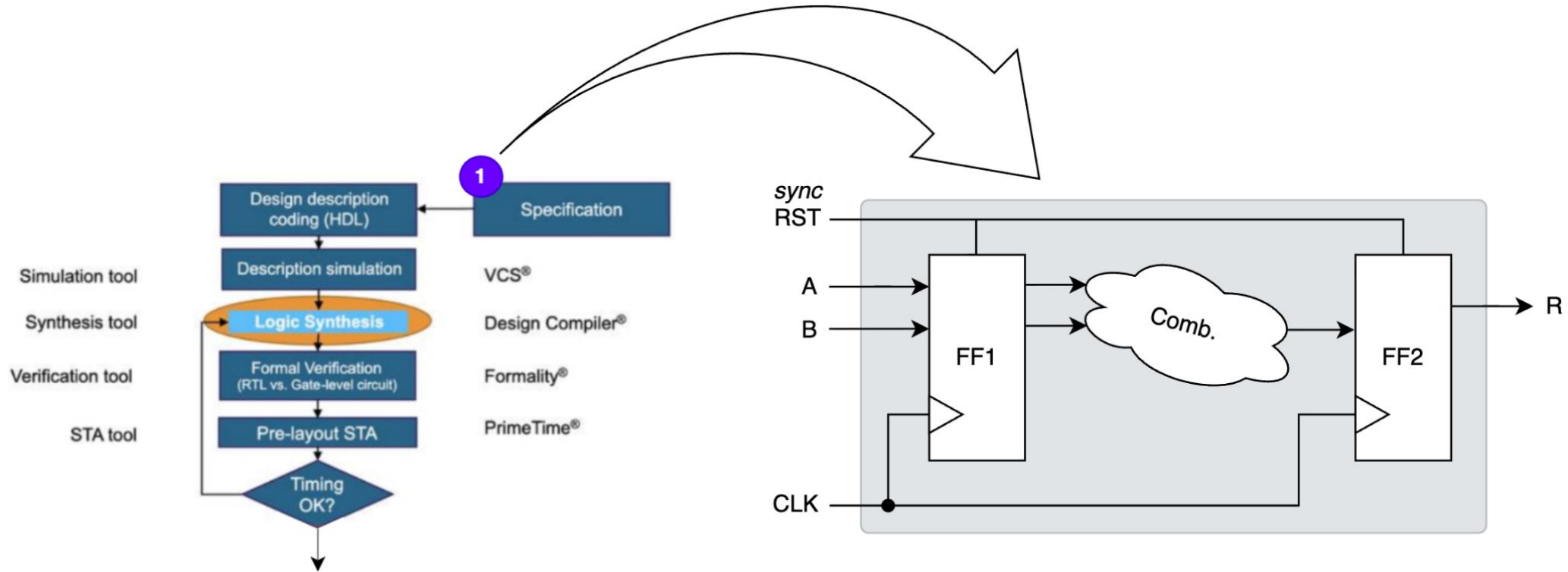


VLSI Flow



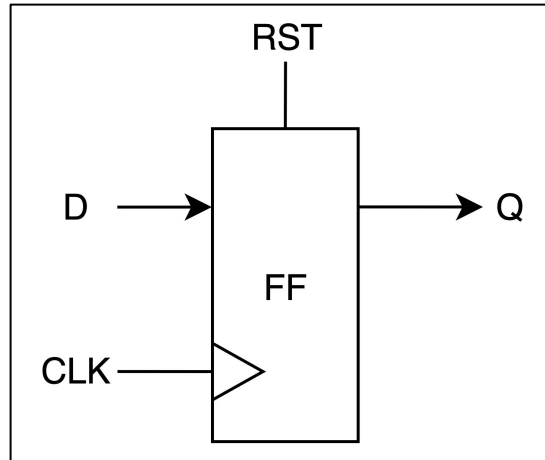
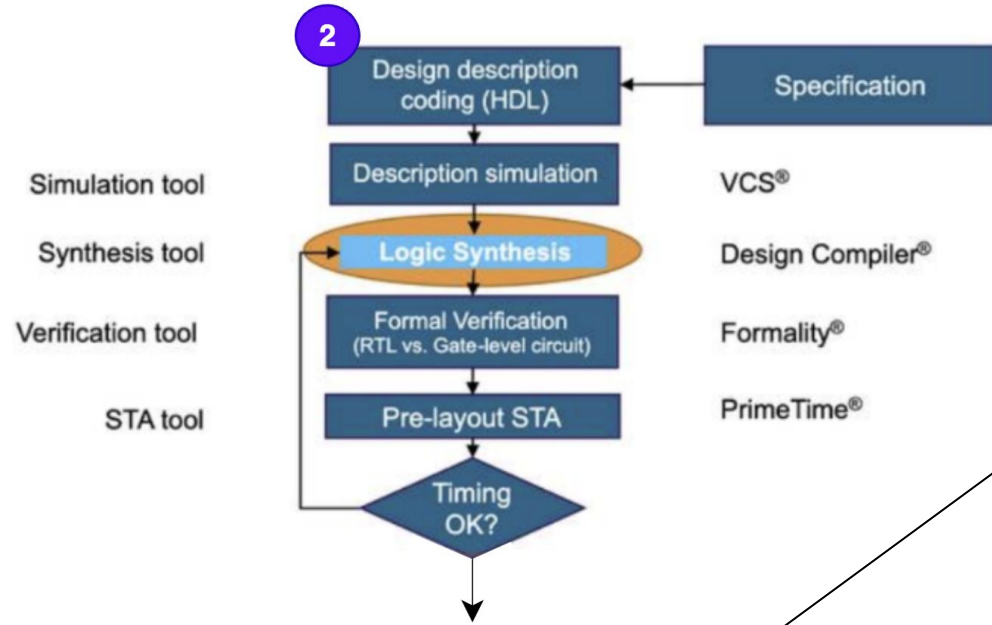
Q) Quantos flip-flops esse circuito tem?

VLSI Flow



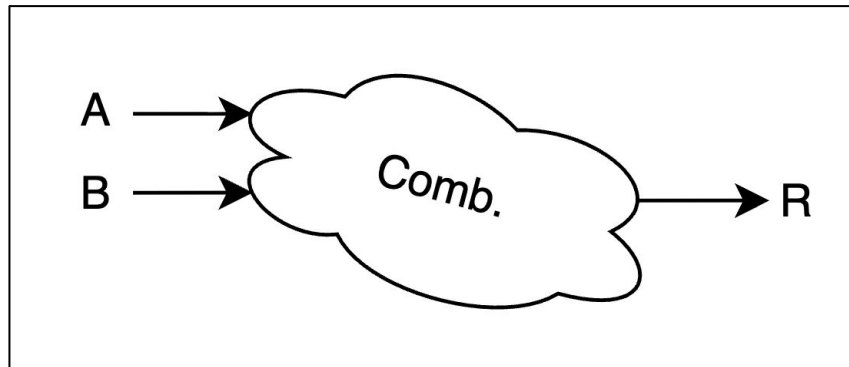
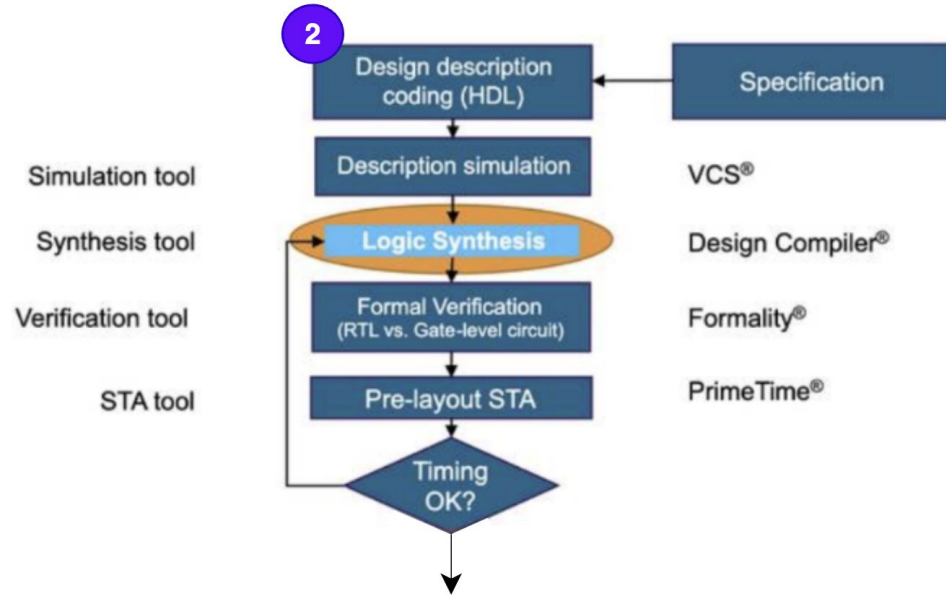
Q) Quantos flip-flops esse circuito tem? $R=3FF$

VLSI Flow



```
1 module ff(  
2     input  logic CLK,  
3     input  logic RST,  
4     input  logic D,  
5     output logic Q  
6 );  
7  
8     always_ff @(posedge CLK) begin  
9         if(RST)  
10            Q <= 1'b0;  
11        else  
12            Q <= D;  
13        end  
14  
15 endmodule
```

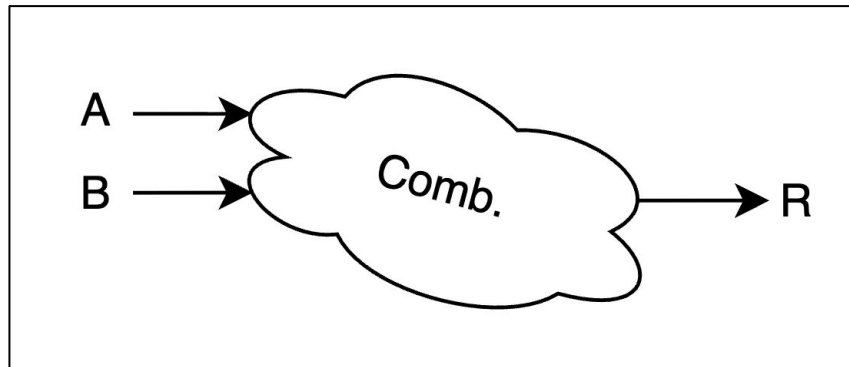
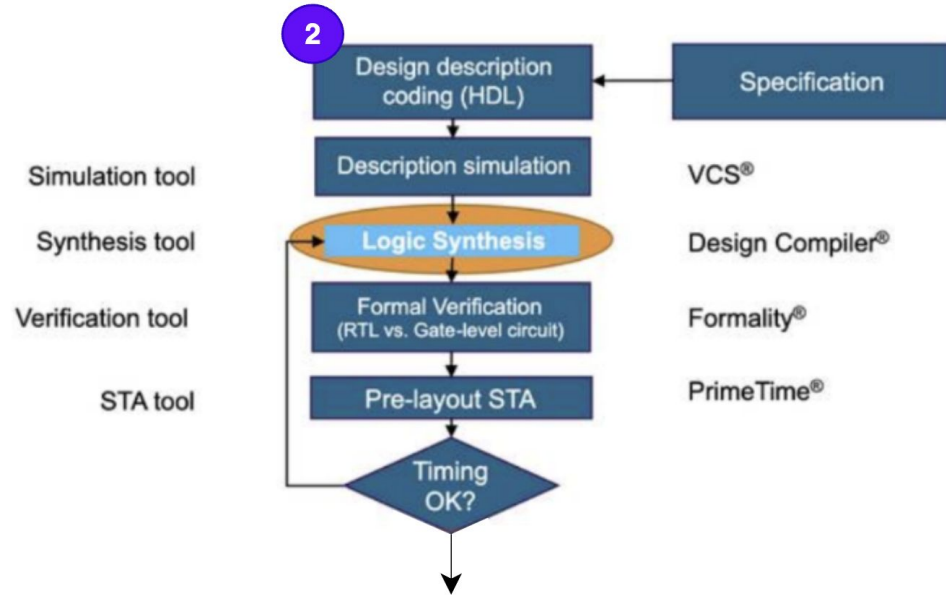
VLSI Flow



```
1 module comb(  
2     input  logic A,  
3     input  logic B,  
4     output logic R  
5 );  
6  
7 always_comb begin  
8     R = A & B;  
9 end  
10  
11 endmodule
```

Q) Qual circuito Comb. está modelando?

VLSI Flow



```
1 module comb(  
2     input  logic A,  
3     input  logic B,  
4     output logic R  
5 );  
6  
7 always_comb begin  
8     R = A & B;  
9 end  
10  
11 endmodule
```

Q) Qual circuito Comb. está modelando? R: AND

MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(  
2      input  logic CLK,  
3      input  logic RST,  
4      input  logic A,  
5      input  logic B,  
6      output logic R  
7  );  
8      // "wires"  
9      logic q_ff1, q_ff2, r_comb;  
10  
11     // FF input 1  
12     ff uu_ff_in1(  
13         .CLK(CLK ),  
14         .RST(RST ),  
15         .D  (A   ),  
16         .Q  (q_ff1)  
17     );  
18  
19     // FF input 2  
20     ff uu_ff_in2(  
21         .CLK(CLK ),  
22         .RST(RST ),  
23         .D  (B   ),  
24         .Q  (q_ff2)  
25     );  
26  
27     // FF  
28     comb uu_comb(  
29         .A(q_ff1 ),  
30         .B(q_ff2 ),  
31         .R(r_comb)  
32     );  
33  
34     ff uu_ff_out(  
35         .CLK(CLK ),  
36         .RST(RST ),  
37         .D  (r_comb),  
38         .Q  (R   )  
39     );  
40 endmodule
```

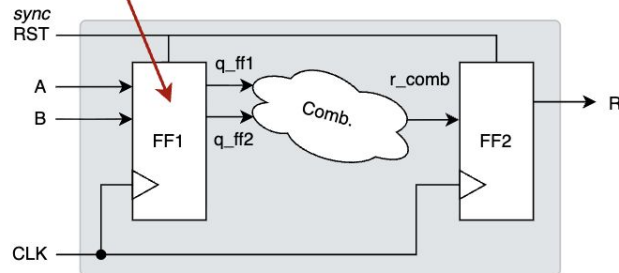
PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets

REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:

<nome_do_modulo> <apelido_do_modulo>



MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MODULOS

```
1  module top(  
2      input  logic CLK,  
3      input  logic RST,  
4      input  logic A,  
5      input  logic B,  
6      output logic R  
7  );  
8      // "wires"  
9      logic q_ff1, q_ff2, r_comb;  
10  
11     // FF input 1  
12     ff uu_ff_in1(  
13         .CLK(CLK ),  
14         .RST(RST ),  
15         .D  (A  ),  
16         .Q  (q_ff1)  
17     );  
18  
19     // FF input 2  
20     ff uu_ff_in2(  
21         .CLK(CLK ),  
22         .RST(RST ),  
23         .D  (B  ),  
24         .Q  (q_ff2)  
25     );  
26  
27     // FF  
28     comb uu_comb(  
29         .A(q_ff1 ),  
30         .B(q_ff2 ),  
31         .R(r_comb)  
32     );  
33  
34     ff uu_ff_out(  
35         .CLK(CLK ),  
36         .RST(RST ),  
37         .D  (r_comb),  
38         .Q  (R  )  
39     );  
40 endmodule
```

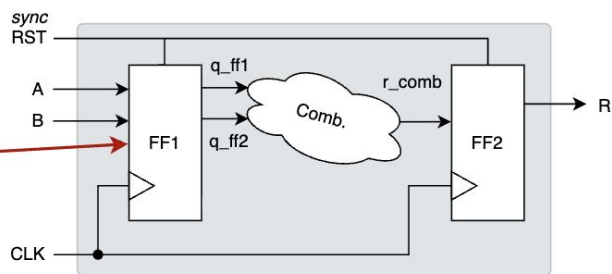
PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets

REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:

<nome_do_modulo> <apelido_do_modulo>



MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MÓDULOS

```
1  module top(  
2      input  logic CLK,  
3      input  logic RST,  
4      input  logic A,  
5      input  logic B,  
6      output logic R  
7  );  
8      // "wires"  
9      logic q_ff1, q_ff2, r_comb;  
10  
11     // FF input 1  
12     ff uu_ff_in1(  
13         .CLK(CLK ),  
14         .RST(RST ),  
15         .D  (A    ),  
16         .Q  (q_ff1)  
17     );  
18  
19     // FF input 2  
20     ff uu_ff_in2(  
21         .CLK(CLK ),  
22         .RST(RST ),  
23         .D  (B    ),  
24         .Q  (q_ff2)  
25     );  
26  
27     // FF  
28     comb uu_comb(  
29         .A(q_ff1 ),  
30         .B(q_ff2 ),  
31         .R(r_comb)  
32     );  
33  
34     ff uu_ff_out(  
35         .CLK(CLK ),  
36         .RST(RST ),  
37         .D  (r_comb),  
38         .Q  (R    )  
39     );  
40 endmodule
```

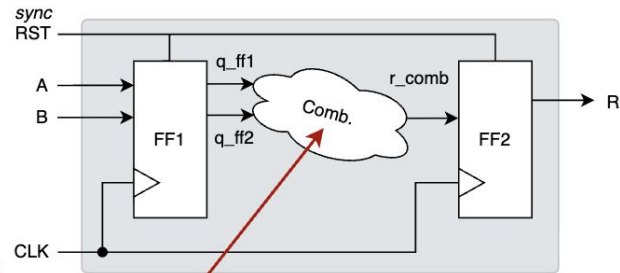
PORTAS: os sinais de interface do modulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets

REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:

<nome_do_modulo> <apelido_do_modulo>



MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MÓDULOS

```
1 module top(  
2     input logic CLK,  
3     input logic RST,  
4     input logic A,  
5     input logic B,  
6     output logic R  
7 );  
8 // "wires"  
9 logic q_ff1, q_ff2, r_comb;  
10  
11 // FF input 1  
12 ff uu_ff_in1(  
13     .CLK(CLK ),  
14     .RST(RST ),  
15     .D (A ),  
16     .Q (q_ff1)  
17 );  
18  
19 // FF input 2  
20 ff uu_ff_in2(  
21     .CLK(CLK ),  
22     .RST(RST ),  
23     .D (B ),  
24     .Q (q_ff2)  
25 );  
26  
27 // FF  
28 comb uu_comb(  
29     .A(q_ff1 ),  
30     .B(q_ff2 ),  
31     .R(r_comb)  
32 );  
33  
34 ff uu_ff_out(  
35     .CLK(CLK ),  
36     .RST(RST ),  
37     .D (r_comb),  
38     .Q (R )  
39 );  
40 endmodule
```

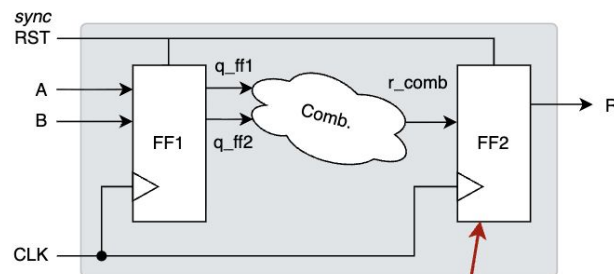
PORTAS: os sinais de interface do módulo é chamado de PORTA

NETS: variáveis internas que funcionam como "fios" são chamados de nets

REGS: variáveis internas que funcionam como registros são chamados de reg

INSTANCIA DE MÓDULOS:

<nome_do_módulo> <apelido_do_módulo>



MÓDULO TOPO (top) INSTANCIA E CONECTA OUTROS MÓDULOS

```
1 module top(  
2     input logic CLK,  
3     input logic RST,  
4     input logic A,  
5     input logic B,  
6     output logic R  
7 );  
8 // "wires"  
9 logic q_ff1, q_ff2, r_comb;  
10  
11 // FF input 1  
12 ff uu_ff_in1(  
13     .CLK(CLK ),  
14     .RST(RST ),  
15     .D (A ),  
16     .Q (q_ff1)  
17 );  
18  
19 // FF input 2  
20 ff uu_ff_in2(  
21     .CLK(CLK ),  
22     .RST(RST ),  
23     .D (B ),  
24     .Q (q_ff2)  
25 );  
26  
27 // FF  
28 comb uu_comb(  
29     .A(q_ff1 ),  
30     .B(q_ff2 ),  
31     .R(r_comb)  
32 );  
33  
34 ff uu_ff_out(  
35     .CLK(CLK ),  
36     .RST(RST ),  
37     .D (r_comb),  
38     .Q (R )  
39 );  
40 endmodule
```

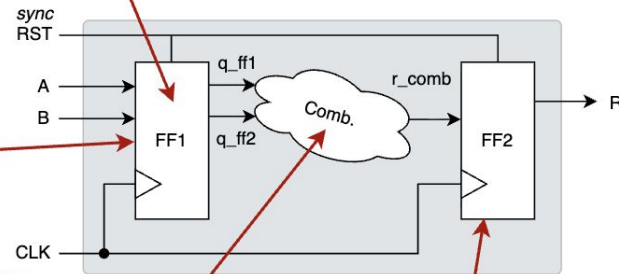
PORTAS: os sinais de interface do modulo é chamado de PORTA

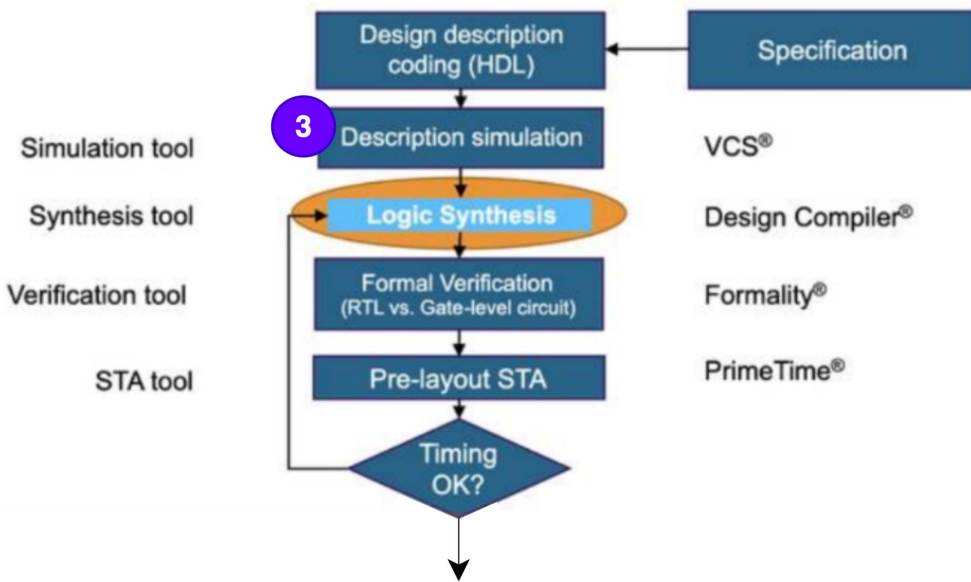
NETS: variáveis internas que funcionam como "fios" são chamados de nets

REGS: variáveis internas que funcionam como registros são chamados de reg

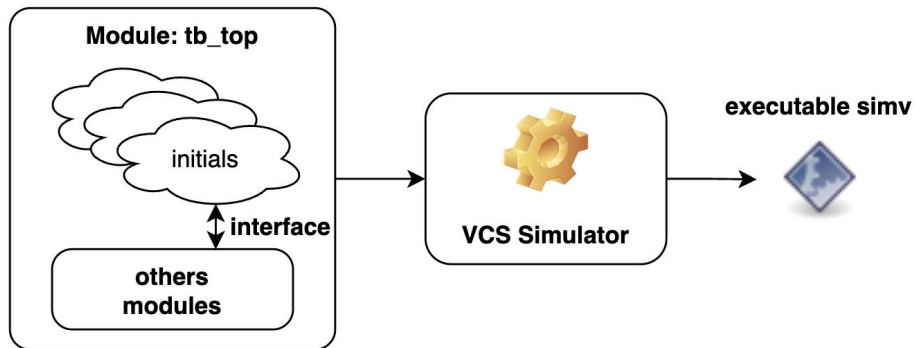
INSTANCIA DE MÓDULOS:

<nome_do_modulo> <apelido_do_modulo>





tb_top... the word top is like the method main()



modules can have input and/or outputs
"initials" drives and monitors the interface

```

1  module tb_top();
2
3      logic clk = 0;
4      logic rst;
5      logic a,b;
6      logic r;
7
8      top uu_top(
9          .CLK(clk),
10         .RST(rst),
11         .A  (a  ),
12         .B  (b  ),
13         .R  (r  )
14     );
15
16     always clk = #5ns ~clk;
17
18     // main
19     initial begin
20         $display("VLSI Flow Lesson");
21         $fsdbDumpvars(); // vcs
22
23         #100ns;
24         $finish();
25     end
26 endmodule
  
```

VLSI Flow

- VCS

```
1  # RTL
2  RTL_FILES = rtl/ff.sv \
3             rtl/comb.sv \
4             rtl/top.sv
5
6  # TB
7  TB_FILES = tb/tb_top.sv
8
9  # VCS
10 VCS_DEBUG = -kdb -debug_access+all
11 VCS_FLAGS = -sverilog -full64 -l ./vcs_logger.log $(VCS_DEBUG) -timescale=1ns/1ps
12
13 vcs:
14 | vcs $(VCS_FLAGS) $(RTL_FILES) $(TB_FILES)
```

```
Parsing design file 'rtl/ff.sv'
Parsing design file 'rtl/comb.sv'
Parsing design file 'rtl/top.sv'
Parsing design file 'tb/tb_top.sv'
Top Level Modules:
    tb_top
TimeScale is 1 ns / 1 ps
Starting vcs inline pass...
1 module and 0 UDP read.
recompiling module tb_top
```

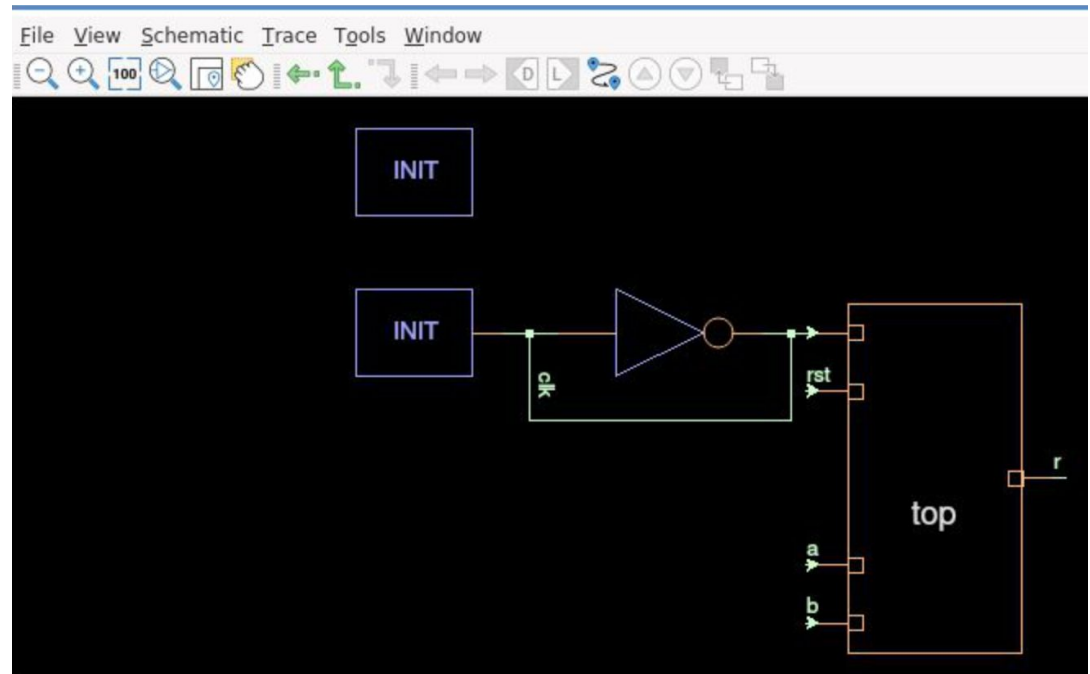
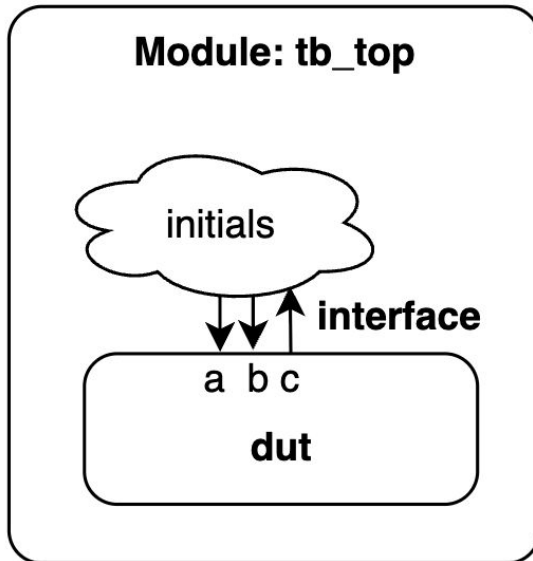


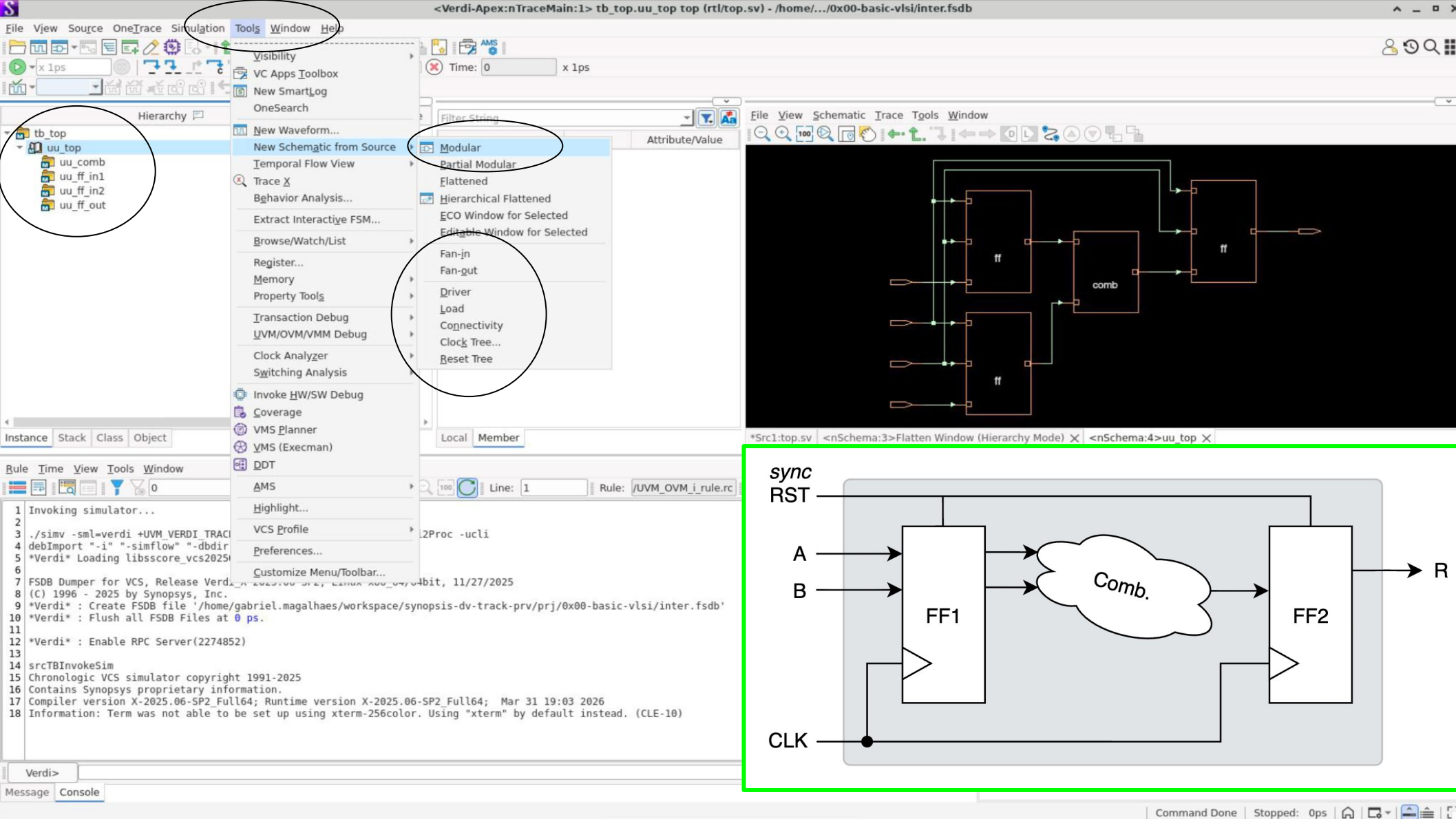
VLSI Flow

- **VCS**
 - **GUI = Verdi: \$./simv -gui**

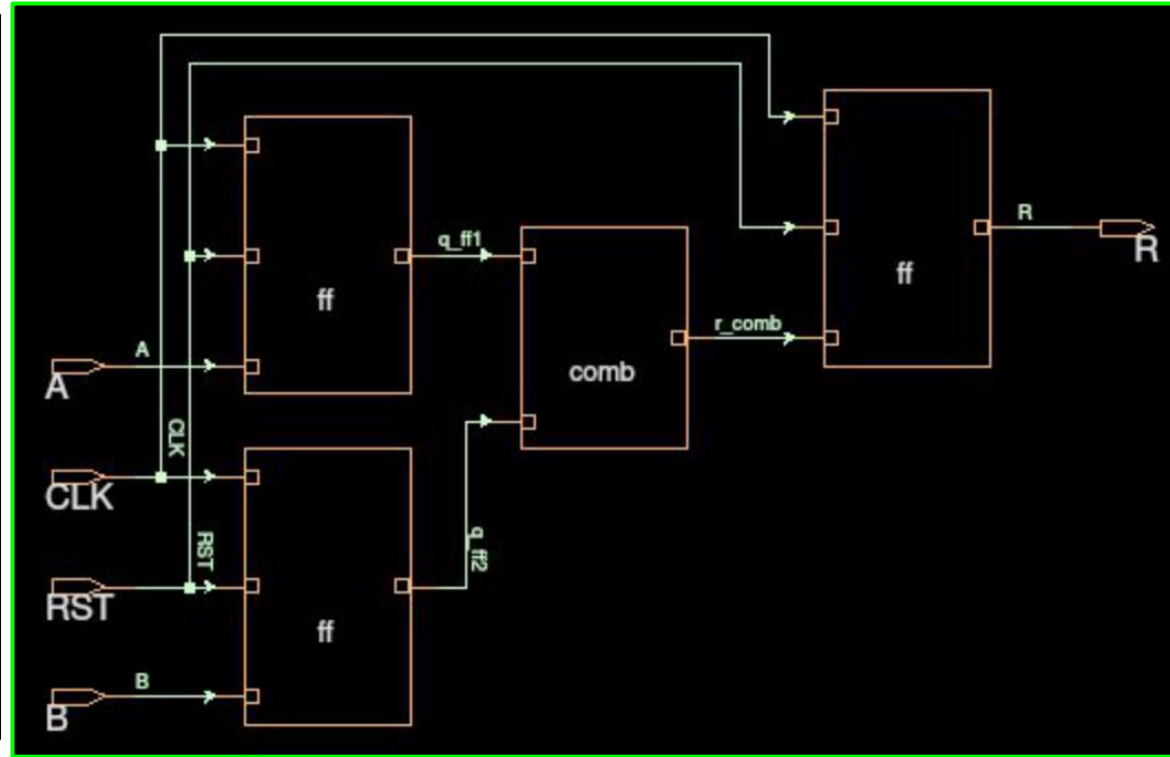
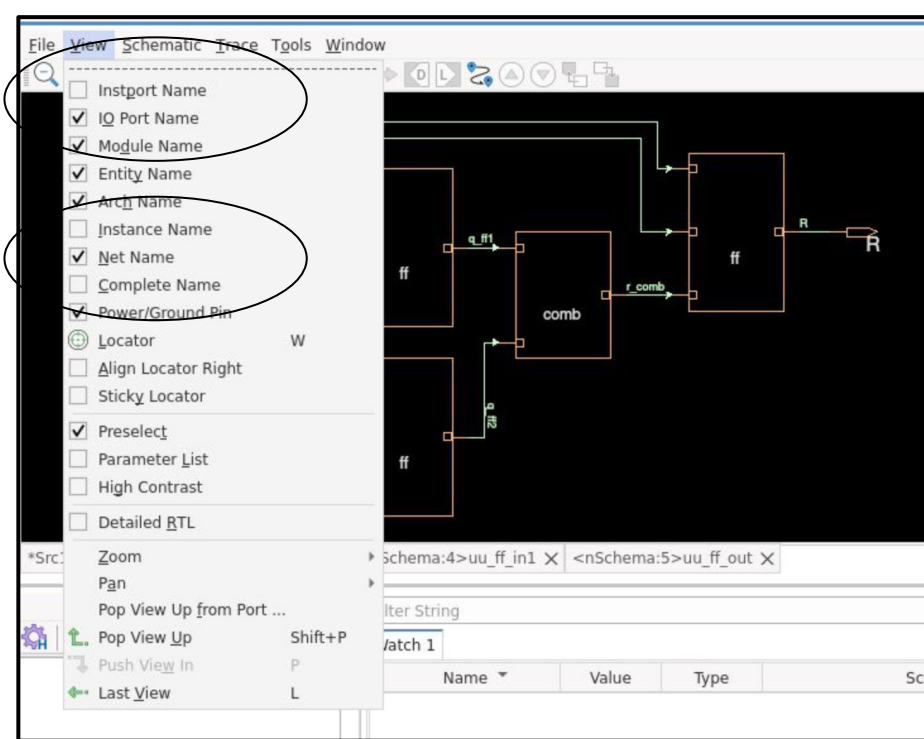
VLSI Flow

- **Verdi (GUI)**
\$./simv -gui



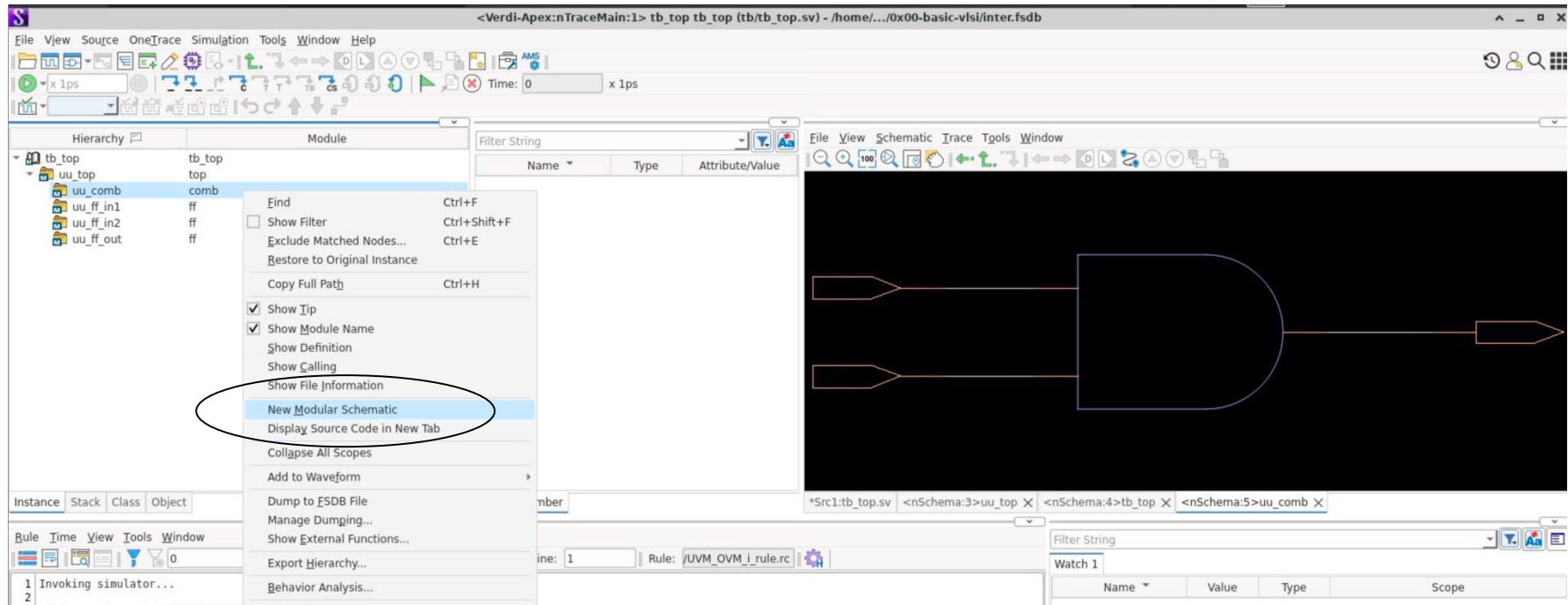


VLSI Flow / PORTs and NETs



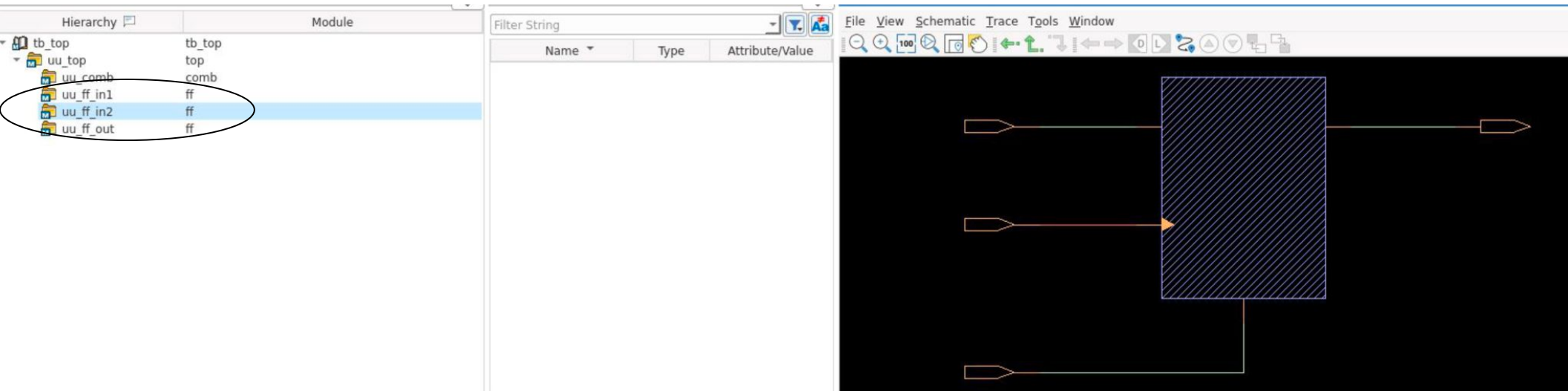
VLSI Flow

- **Verdi**
 - **comb = AND**



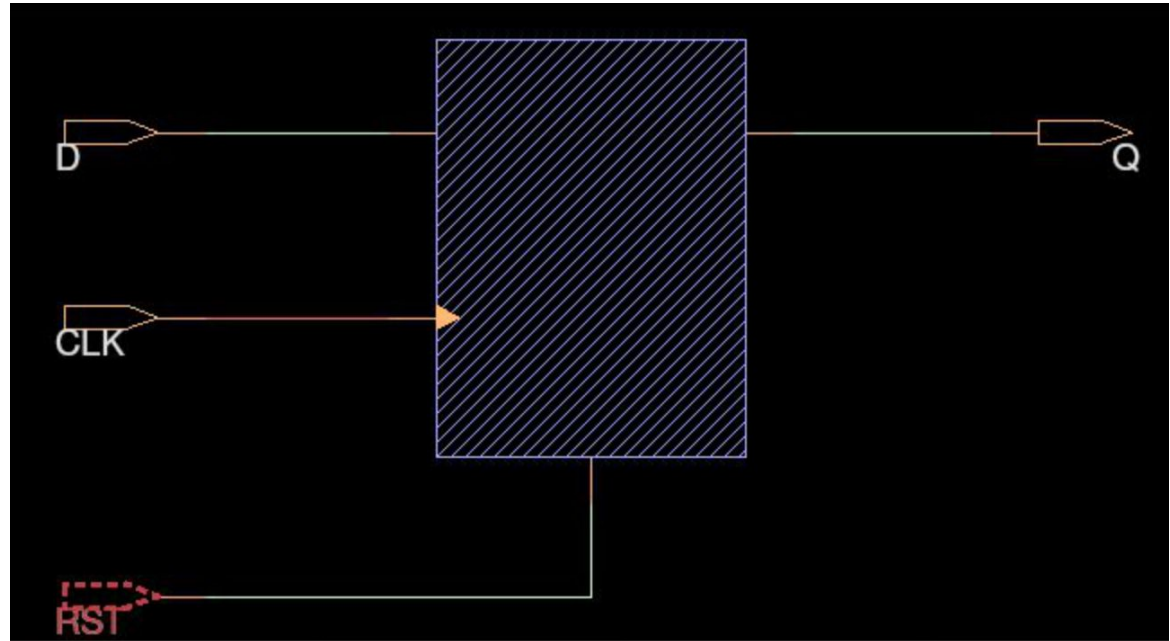
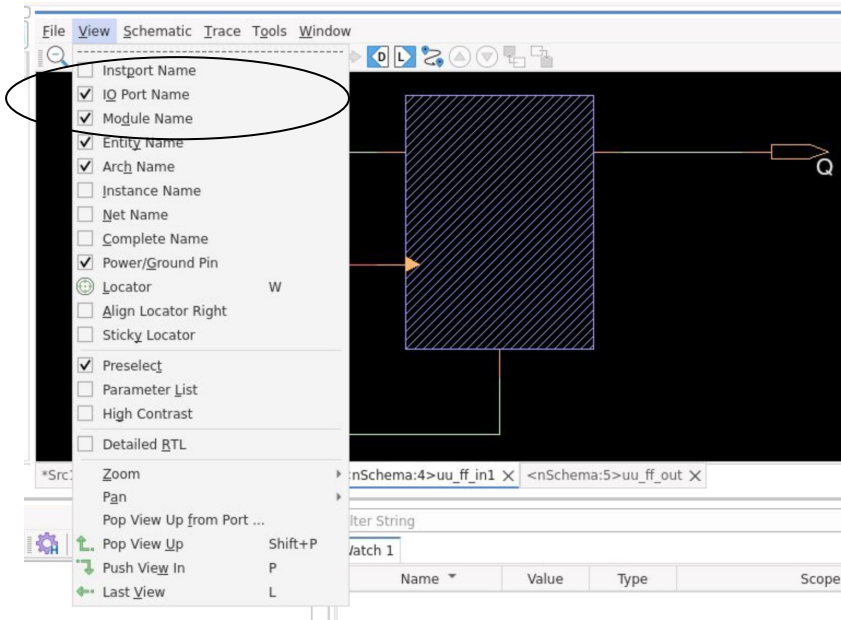
VLSI Flow

- **Verdi**
 - **ff = Flip-Flop**



VLSI Flow

- **Verdi (PORTs)**
 - **ff = Flip-Flop**





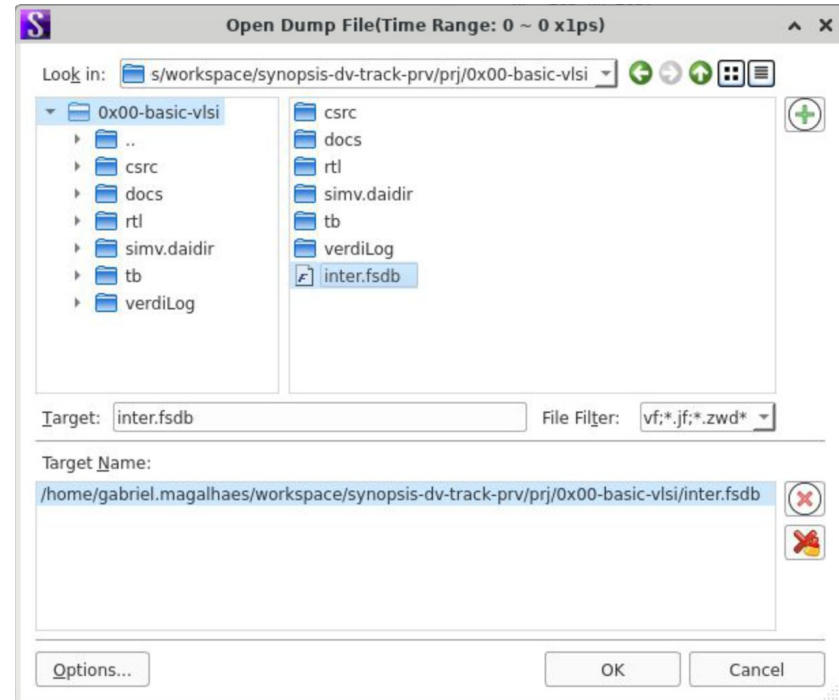
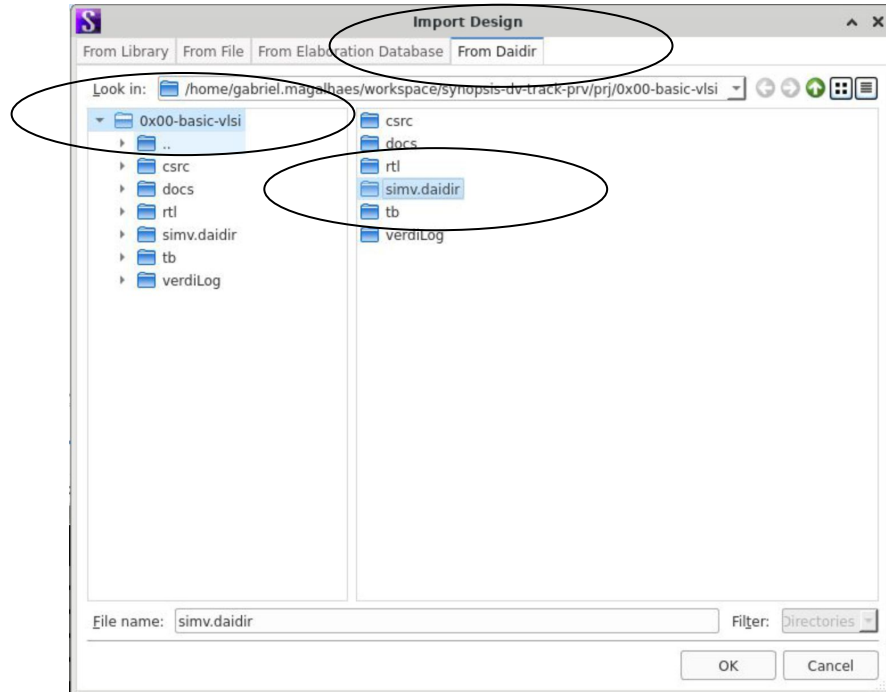
VLSI Flow

- **Verdi (PORTs)**

E como rodar a simulação e ver formas de onda?

VLSI Flow

- **Verdi: Import Design => Open Dump (in waveform)**



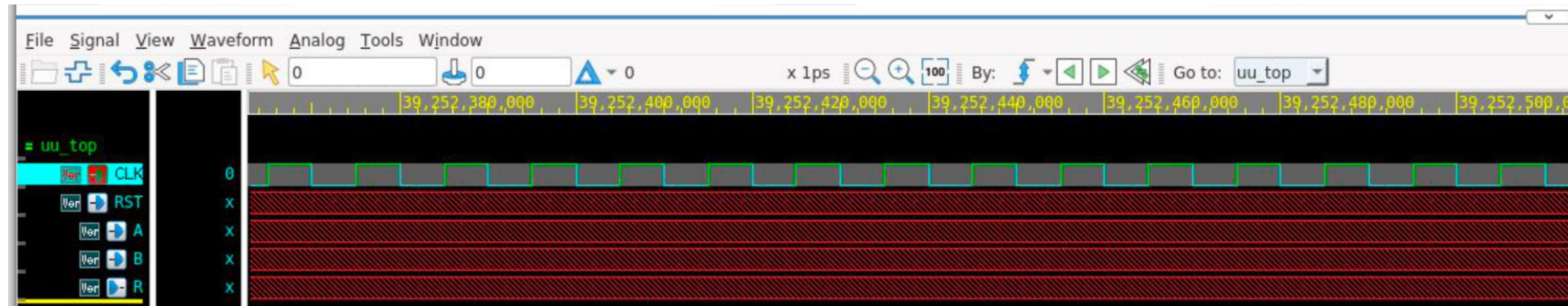
VLSI Flow

- **Verdi:**
 - Arraste o modulo top para dentro da Waveform
 - Clique em "Play Simulation" 

VLSI Flow

- **Verdi:**
 - Arraste o modulo top para dentro da Waveform
 - Clique em "Play Simulation" 

Q) Os sinais estão em X, o que significa?



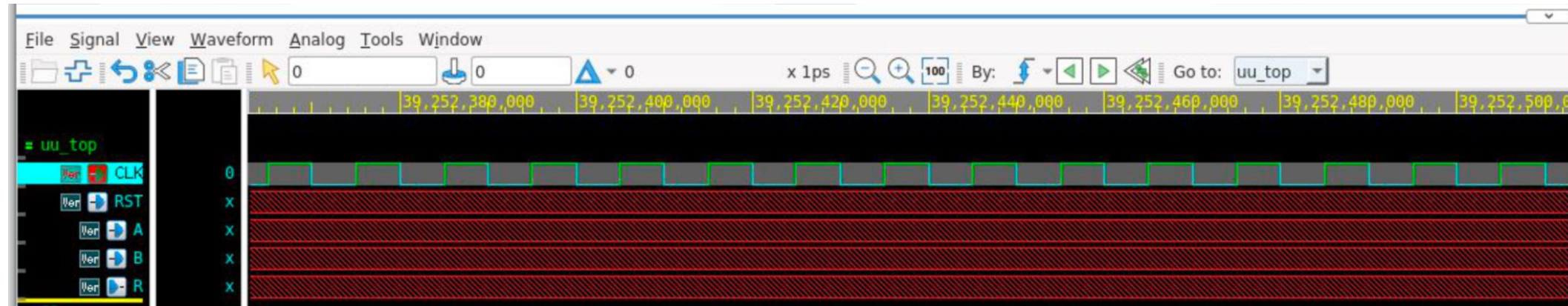
VLSI Flow

- **Verdi:**

- Arraste o modulo top para dentro da Waveform
- Clique em "Play Simulation" 

4-state Value
0
1
Z
X

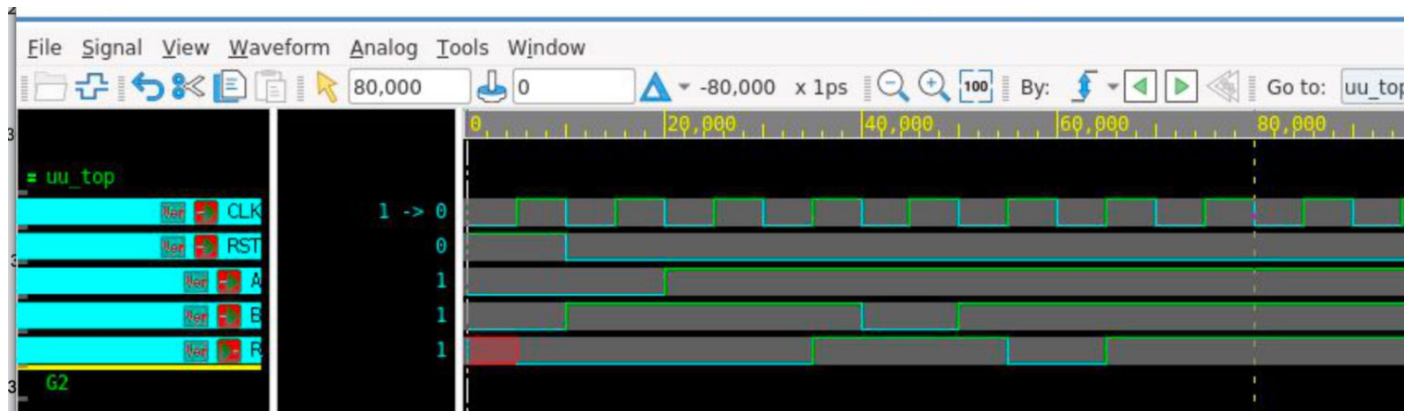
Q) Os sinais estão em X, o que significa?



VLSI Flow

- **DICA:**
 - Todos os comandos ficam salvos em verdiLog/verdi.cmd
- Copie verdi.cmd e rode o verdi:
\$ cp verdiLog/verdi.cmd . # só faz 1 vez
\$ verdi -play verdi.cmd

VLSI Flow



```
18 // main
19 initial begin
20     $display("VLSI Flow Lesson");
21     $fsdbDumpvars(); // vcs
22
23     // init signals
24     a = 0;
25     b = 0;
26
27     // do reset
28     rst = 1;
29     @(negedge clk);
30     rst = 0;
31
32     repeat(5) begin
33         a = $random();
34         b = $random();
35         @(negedge clk);
36     end
37
38     #100ns;
39     $finish();
40 end
41 endmodule
```

Hierarchy

	Module
tb_top	tb_top
uu_top	top
uu_comb	comb
uu_ff_in1	ff
uu_ff_in2	ff
uu_ff_out	ff

Filter String

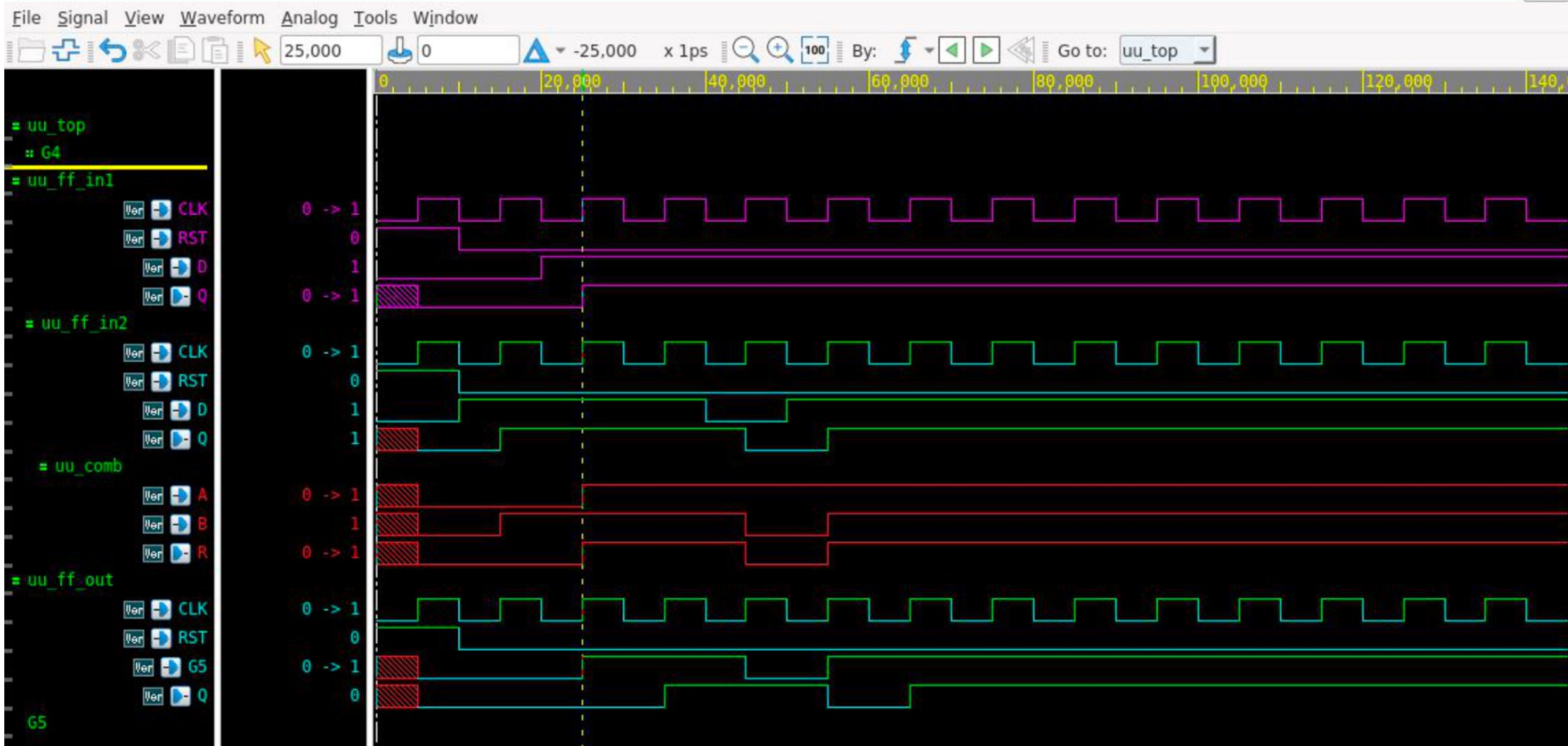
Name	Type	Attribute/Value
------	------	-----------------

Instance Stack Class Object

Local Member

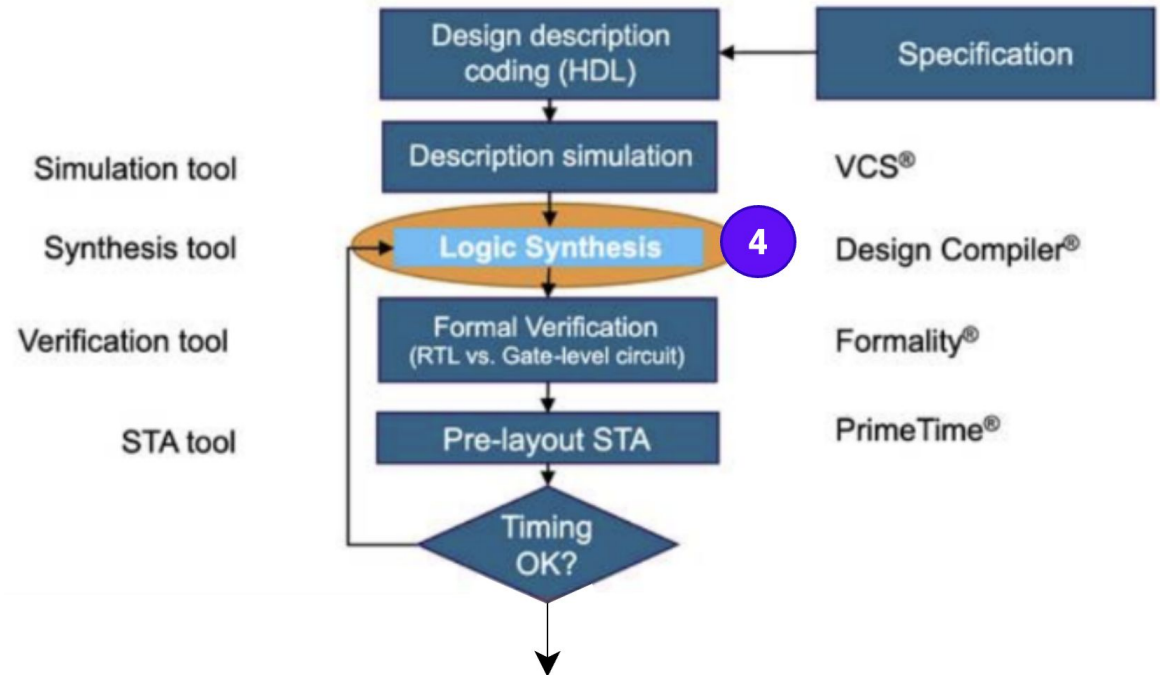
```
29 @(negedge clk);
30 rst = 0;
31
32 repeat(5) begin
33   a = $random();
34   b = $random();
35   @(negedge clk);
36 end
37
38 #100ns;
```

\$ verdi -play verdi.cmd



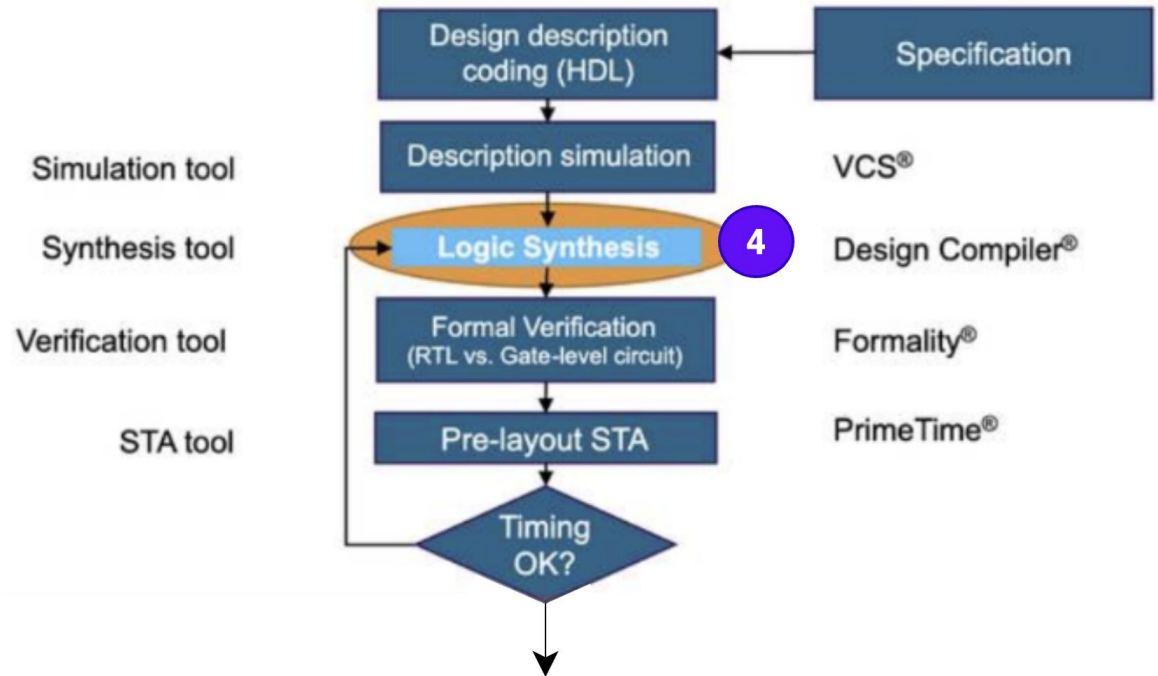
VLSI Flow

- Descrição HDL
- Testbench
- Síntese



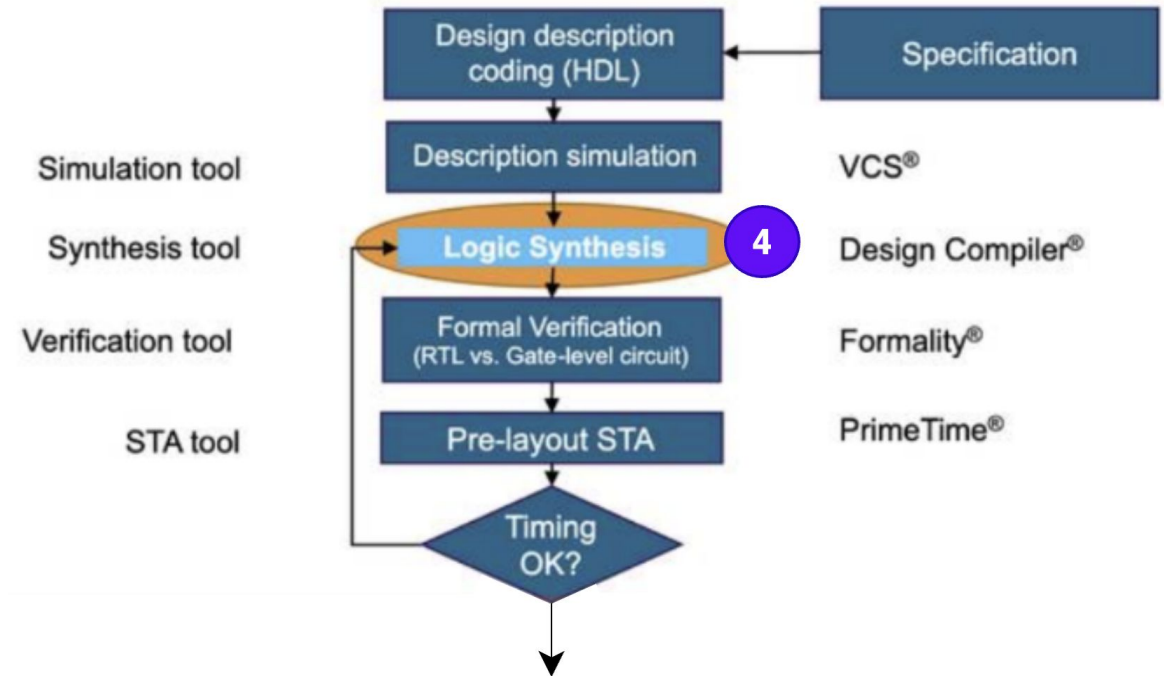
VLSI Flow

- Descrição HDL
- Testbench
- Síntese



Q) A síntese faz e depende de que?

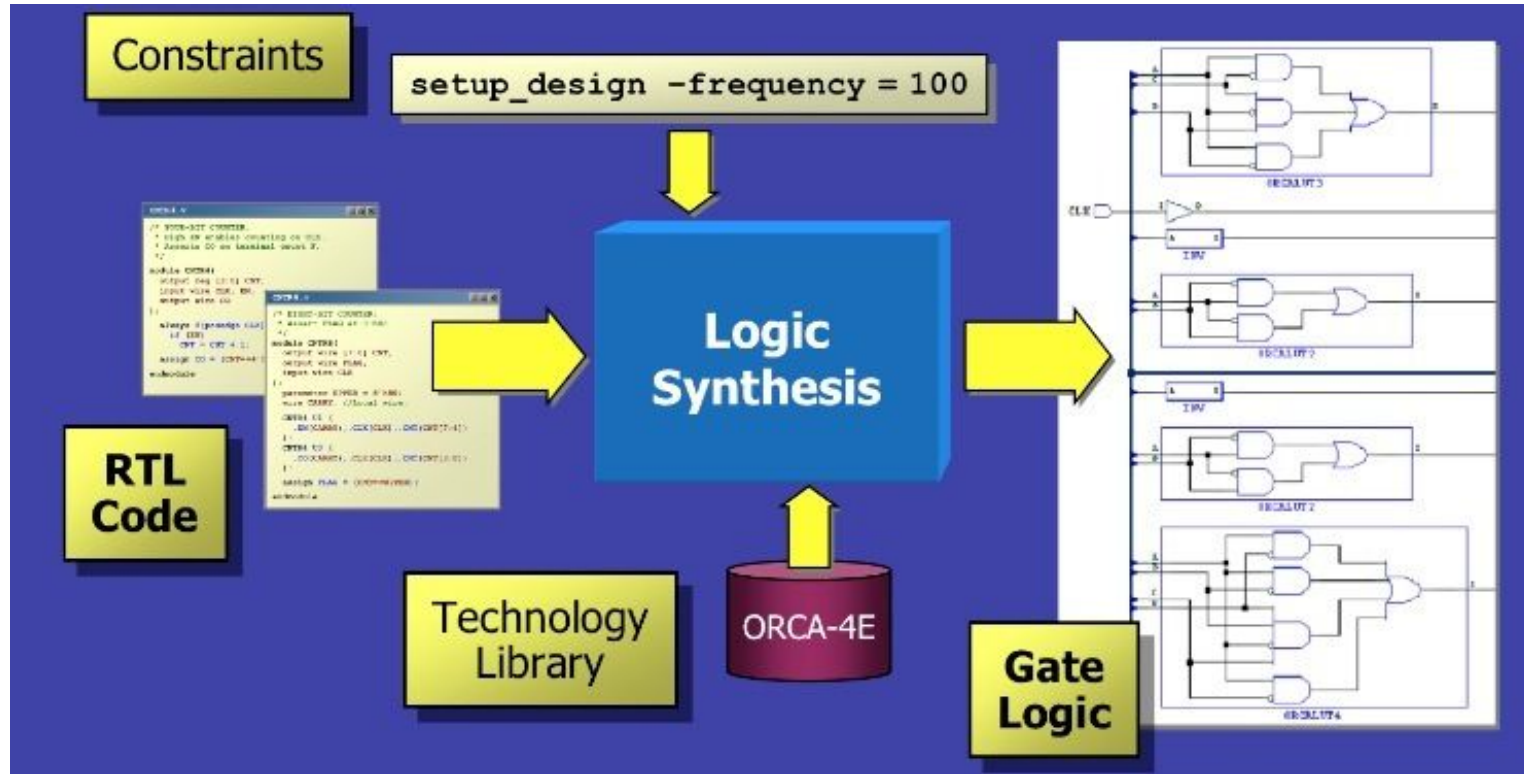
- 



Q) A síntese faz e depende de que?

R: Transforma RTL em netlist Lógica; Depende do PDK + Constraints

VLSI Flow





VLSI Flow

=> PARAMOS AQUI ...

EXECUÇÃO:



APOIO:



INICIATIVA:

